TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

BUỔI 12 · THỰC HÀNH · ★ NỘP BT2

Workshop tích hợp hệ thống và Demo nhanh

★ NỘP BÀI TẬP 2 trước khi kết thúc buổi — 20% điểm học phần
Kết thúc Giai đoạn 2

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

MỤC TIÊU BUỔI HỌC

Ba việc, việc thứ ba là bắt buộc

1

HOÀN THIỆN tích hợp: luồng chính chạy end-to-end, không lỗi treo

[SE] thêm ≥ 1 tính năng phụ nếu còn thời gian · [DA] pipeline chạy tự động bằng một lệnh · [AI] đóng gói mô hình thành API demo gọi được. **Ưu tiên: luồng chính chạy ổn định hơn là thêm tính năng.**

2

DEMO nhanh 3 phút và để giảng viên chạy thử repo

Giảng viên clone repo của em theo tag v0.2 và làm đúng theo README ngay tại lớp, rồi hỏi em về một hàm bất kỳ. Nếu thầy không chạy được (C1) thì mọi mục khác không kiểm chứng được.

3

★ NỘP BÀI TẬP 2 qua VLU E-learning trước khi kết thúc buổi

1 file PDF (BT2_<MSSV>_<Họ tên không dấu>.pdf) **+ link GitHub repo có tag v0.2.** Nộp trễ: trừ 10% mỗi ngày; trễ quá 3 ngày tính 0 điểm thành phần này VÀ không đủ điều kiện dự thi cuối kỳ.

NỘI DUNG NỘP BÀI TẬP 2 · 20% ĐIỂM HỌC PHẦN

Nộp gì cho Bài tập 2 — và nộp thế nào

1 · Hình thức và thời hạn

- Nộp qua **VLU E-learning**: **1 file PDF + link GitHub repo có tag v0.2**.
- Tên file: **BT2_<MSSV>_<Họ tên không dấu>.pdf** — một file duy nhất, không nộp .docx.
- Hạn: **trước khi kết thúc buổi 12**.
- Nộp bù: **tối đa 3 ngày**, trừ **10% điểm mỗi ngày** trễ.
- Quá 3 ngày: **0 điểm** thành phần và **không đủ điều kiện dự thi** cuối kỳ.
- Repo để **PUBLIC** — giảng viên không nhận lời mời collaborator. Vì vậy repo tuyệt đối không chứa .env hay secret.

2 · Năm thành phần trong repo (tag v0.2)

- **TP1 – Module cốt lõi** chạy được, tách lớp. [SE] ≥ 2 thực thể đủ CRUD · [DA] pipeline 3 pha, ≥ 5 hàm kiểm chất lượng · [AI] script huấn luyện + mô hình cơ sở.
- **TP2 – ≥ 3 kiểm thử** thuộc ba loại (luồng đúng · biên · dữ liệu sai), PASS bằng một lệnh.
- **TP3 – Module mở rộng** và phần trình bày kết quả, đủ ba trạng thái.
- **TP4 – Hệ thống chạy được** theo README: tệp khoá, .env.example, dữ liệu mẫu.
- **TP5 – Tài liệu:** README.md · docs/thay-doi-thiet-ke.md · docs/ai-disclosure.md.

3 · Cấu trúc file PDF (6–9 trang)

- **Trang bìa:** link repo + tag v0.2, track, mã luồng nghiệp vụ, họ tên – MSSV – lớp.
- **Mục 1** Tóm tắt hệ thống · **Mục 2** Kiến trúc đã hiện thực.
- **Mục 3** Module cốt lõi — **ảnh chụp** chạy được.
- **Mục 4** Bộ kiểm thử — **ảnh chụp** kết quả PASS.
- **Mục 5** Trình bày kết quả — **ảnh chụp** đủ ba trạng thái.
- **Mục 6** Cách dựng lại hệ thống · **Mục 7** Thay đổi so với BT1.
- **Phụ lục (bắt buộc):** bảng khai báo sử dụng công cụ AI.

ẢNH CHỤP KẾT QUẢ CHẠY THẬT là minh chứng bắt buộc cho mục 3, 4, 5 của PDF: thấy rõ lệnh đã chạy và kết quả (terminal, giao diện, test PASS). Mục nào không có ảnh chụp thì tính là **không có minh chứng** — chụp ngay khi chạy được, đừng để tới lúc nộp.

NHỊP BUỔI HỌC

Bốn chặng: mở đầu, hoàn thiện, demo và chạy thử, nộp bài

1

Chặng 1
Mở đầu

GV chiếu bảng tiêu chí kỹ thuật BT2 (bốn mục CỐT LÕI) và thứ tự ưu tiên khi thiếu thời gian.

2

Chặng 2
Hoàn thiện

Hoàn thiện tích hợp, sửa lỗi còn lại, dọn repo theo bảy việc.

GV bắt đầu đi lớp clone-and-run ngay trong chặng này.

3

Chặng 3
Demo + chạy thử

Demo nhanh 3 phút/SV.

GV clone theo tag v0.2, chạy theo README, hỏi một hàm, tick biểu chấm tại chỗ.

4

Chặng 4
NỘP BÀI

Đẩy tag v0.2, xuất PDF, nộp qua VLU E-learning.

Kiểm tra đã nộp thành công. GV chốt danh sách đã nộp.

THỨ TỰ ƯU TIÊN KHI THIẾU THỜI GIAN: (1) bốn mục CỐT LÕI — chạy được theo README (C1), tách lớp (C2), ≥ 3 test ba loại PASS (C4), chạy lại không nhân đôi (C5); (2) dọn repo, bỏ secret, đẩy tag; (3) PDF đủ 7 mục + khai báo AI; (4) mới tới thêm tính năng — việc ÍT ĐIỂM NHẤT.

Bảy việc dọn repo — làm trước khi gọi giảng viên

- [ ] **1 — git status sạch:** không thấy node_modules/, .venv/, __pycache__/, .env hay tệp dữ liệu lớn. Nếu có: bổ sung .gitignore rồi git rm -r --cached <thư mục>.
- [ ] **2 — Tìm secret:** grep -rniE "password|api_key|secret" src/ — không có giá trị thật. Lỡ commit thì ĐỔI KHOÁ trước, rồi mới sửa repo.
- [ ] **3 — Tệp khoá đã commit:** package-lock.json, hoặc requirements.txt ghi == (tạo từ venv sạch).
- [ ] **4 — .env.example** liệt kê đủ TÊN biến, giá trị là giá trị mẫu, KHÔNG có giá trị thật.
- [ ] **5 — README:** đủ phiên bản runtime, mỗi bước một lệnh sao chép được, có lệnh chạy test.
- [ ] **6 — CLONE VÀO THƯ MỤC MỚI** và làm đúng từng bước README. Việc quan trọng nhất trong bảy việc.
- [ ] **7 — Tag và đẩy:** git tag -a v0.2 -m "Bai tap 2" → git push origin main → git push origin v0.2 → mở trang …/releases/tag/v0.2 để kiểm.
- [ ] **+ Dọn mã:** xoá print/console.log gỡ lỗi; catch rỗng; tệp docs/bug-log.md đã có ít nhất một mục.

Việc 6 mất 5 phút và quyết định mục CỐT LÕI C1 của BT2. Lệnh git push origin main KHÔNG đẩy tag — phải đẩy tag riêng, nếu không giảng viên không lấy được bài.

KHI PHÉP THỬ THƯ MỤC TRỐNG THẤT BẠI

Đọc thông báo lỗi — biết README hay repo thiếu gì

Thông báo trên máy “sạch”	Nghĩa là	Sửa thế nào (rồi chạy lại phép thử)
Cannot find module 'dotenv' · ModuleNotFoundError: No module named 'pandas'	Thư viện có trên máy em (cài toàn cục) nhưng không có trong tệp khoá	npm install <gói> / pip install trong venv rồi cập nhật tệp khoá; commit
DB_PASSWORD is undefined · KeyError: 'DWH_URL'	README thiếu bước tạo .env, hoặc .env.example thiếu biến	Thêm bước cp .env.example .env vào README; bổ sung biến còn thiếu
relation "tickets" does not exist	Chưa chạy migration — trên máy em bảng đã có từ trước	Thêm lệnh migration + seed vào README (hoặc chạy tự động khi khởi động)
Màn hình / dashboard trống trơn, không lỗi	Không có dữ liệu mẫu	Thêm seed hoặc tệp dữ liệu mẫu nhỏ trong repo; ghi lệnh nạp
Test PASS trên máy em, FAIL trên máy thầy	Test phụ thuộc dữ liệu hoặc tệp có sẵn trên máy em	Test tự dựng dữ liệu nó cần (fixture) hoặc dùng đối tượng giả
Lần chạy thứ hai báo trùng khoá / số dòng nhân đôi	Pipeline hoặc seed chưa chạy lại được (C5)	Nạp theo khoá nghiệp vụ (ON CONFLICT) hoặc xoá-rồi-nạp theo lô
fatal: Remote branch v0.2 not found	Chưa đẩy tag lên GitHub	git push origin v0.2; kiểm trang releases

Quy trình gỡ bốn bước của buổi 11 áp dụng nguyên vẹn: đọc thông báo (tái lập) → xác định thiếu bước nào (khoanh vùng) → sửa README/repo (sửa nguyên nhân) → chạy lại phép thử từ đầu (xác nhận).

CHẶNG 3 · KHUNG DEMO NHANH 3 PHÚT

Bốn điều phải cho thấy trong 3 phút

1

Luồng chính chạy end-to-end

Chạy thật, không nói suông. Từ điểm bắt đầu tới điểm kết thúc, một mạch.

2

Một trường hợp lỗi được xử lý đúng

Nhập dữ liệu sai: hệ thống báo lỗi rõ ràng, không sập (C7).

3

Lệnh chạy test hiện số test PASS

Chạy lệnh test ghi trong README, cho thấy ≥ 3 test thuộc ba loại PASS (C4).

4

Một lỗi em đã gặp và cách em gỡ

Kể theo bốn bước: tái lập, khoanh vùng, sửa nguyên nhân, xác nhận — mở mục tương ứng trong docs/bug-log.md.

Lớp đông không đủ thời gian cho tất cả: 10–12 sinh viên demo trước lớp, số còn lại demo riêng khi giảng viên tới bàn. Bấm đồng hồ đúng 3 phút.

Bốn mục CỐT LÕI và ba mục quan sát — kiểm ngay tại bàn

Mục kiểm chứng	Lệnh hoặc thao tác	Đạt khi
C1 · CỐT LÕI **Clone và chạy theo README**	git clone --branch v0.2 <url> thu-nghiem, rồi làm đúng từng bước README	Hệ thống dựng lại được từ thư mục trống, không phải hỏi thêm sinh viên bước nào
C2 · CỐT LÕI **Tách lớp đúng**	Mở tệp service (hoặc transform, features)	Ba lớp ở ba nơi; service không có req, res, mã trạng thái HTTP
C4 · CỐT LÕI **Test chạy PASS**	Chạy lệnh test ghi trong README (npm test / pytest / mvn test)	≥ 3 test thuộc ba loại (luồng đúng · biên · dữ liệu sai) PASS bằng một lệnh
C5 · CỐT LÕI **Chạy lại lần thứ hai**	Chạy lại migration/seed, pipeline hoặc script huấn luyện	Không lỗi, không nhân đôi dữ liệu; [AI] hai tệp chỉ số giống nhau
C7 · Dữ liệu sai	Gọi một chức năng với dữ liệu sai	Từ chối có thông báo rõ, không sập, không trả lỗi thô của thư viện
Suy giảm có kiểm soát	Tắt một thành phần phụ trợ rồi thử lại nghiệp vụ chính	Nghiệp vụ chính vẫn chạy, hoặc báo lỗi rõ ràng — không treo
Lịch sử commit (tiêu chí 4)	git log --oneline | head -20	Commit nhỏ, đều theo thời gian, thông điệp mô tả được việc đã làm (CLO5)

Thiếu MỘT mục CỐT LÕI (C1, C2, C4, C5) → tiêu chí Chất lượng kỹ thuật về mức Yếu, vì các mục còn lại không thể bù. Đây là lý do "clone thử vào thư mục mới" quan trọng hơn mọi tính năng em thêm vào.

Mục kiểm chứng riêng của từng chuyên ngành

Track	Lệnh hoặc thao tác	Đạt khi
[SE]	curl hoặc Postman gọi CRUD của 2 thực thể; tắt backend rồi mở giao diện; lọc theo điều kiện không có kết quả	Đủ CRUD cho ≥ 2 thực thể (S1); có migration + seed (S3); ≥ 3 màn hình gọi API thật, đủ ba trạng thái (S4)
[DA]	Chạy pipeline bằng một lệnh, rồi chạy LẦN THỨ HAI; đếm dòng bảng đích; mở bảng reject và dashboard	Ba pha tách riêng (D1); ≥ 5 hàm kiểm chất lượng (D2); bảng reject có lý do (D3); ≥ 2 biểu đồ có mốc tham chiếu (D4)
[AI]	Chạy script huấn luyện hai lần, so sánh hai tệp chỉ số; đọc thứ tự chia dữ liệu và fit	Chia dữ liệu trước khi fit (A1); có mô hình cơ sở (A2); chọn theo validation (A3); chỉ số theo nhãn + confusion matrix (A4)

Mỗi sinh viên chỉ cần chứng minh mục của track mình. Mục nào không chứng minh được tại lớp hoặc trong PDF thì tính là KHÔNG ĐẠT, dù thực tế em có làm.

BỘ BA CÂU HỎI CHUẨN · GIẢNG VIÊN CHỌN NGẪU NHIÊN MỘT HÀM

Chuẩn bị trả lời bằng mã của chính em

1. Vì sao em chọn cách này mà không phải cách kia?

- **Trả lời yếu:** “Em thấy trên mạng làm vậy.” · “AI gợi ý ạ.”
- **Trả lời tốt:** “Em nạp theo khoá nghiệp vụ bằng ON CONFLICT để chạy lại không nhân đôi (C5). Cách xoá-rồi-nạp cũng được nhưng mất lịch sử cập nhật.”
- Nêu được ít nhất MỘT phương án khác và vì sao không chọn.

2. Nếu dữ liệu tăng gấp mười thì chỗ nào hỏng trước?

- **Trả lời yếu:** “Chắc là vẫn chạy được ạ.”
- **Trả lời tốt:** “Màn hình danh sách đang tải hết phiếu không phân trang — sẽ chậm trước. Em sẽ thêm LIMIT/OFFSET và chỉ mục trên due_date.”
- Chỉ ra MỘT điểm nghẽn cụ thể và cách đo.

3. Chỉ cho thầy dòng mã chịu trách nhiệm cho việc này

- **Trả lời yếu:** giải thích chung chung bằng lời, không mở tệp.
- **Trả lời tốt:** mở src/services/ticket.service.js, chỉ dòng kiểm QT-05 và test tương ứng trong tests/.
- Câu quan trọng nhất. Không tìm được dòng đó là dấu hiệu chưa hiểu phần mình nộp.

Không giải thích được mã mình nộp → phần đó tính là **KHÔNG CÓ MINH CHỨNG** (mục 4.4 đề BT2). Phần có AI hỗ trợ cũng phải giải thích được từng dòng — đó là điều kiện của chính sách liêm chính AI.

CHECKLIST NỘP BÀI TẬP 2 · ĐỐI CHIẾU TRƯỚC KHI BẤM NỘP

Mười mục hình thức — tránh mất điểm không đáng

- [ ] **Một file PDF duy nhất**, tên BT2_<MSSV>_<Họ tên không dấu>.pdf — không nộp .docx, không nhiều tệp rời.
- [ ] Trang bìa có **link GitHub repo và tên tag v0.2**, track, mã luồng nghiệp vụ.
- [ ] PDF đủ 7 mục: tóm tắt · kiến trúc thực tế · module cốt lõi · kiểm thử · trình bày kết quả · cách dựng lại · thay đổi so với BT1.
- [ ] **Ảnh chụp kết quả chạy thật** trong PDF: mục 3 module chạy · mục 4 test PASS · mục 5 đủ **ba trạng thái**.
- [ ] Phụ lục **bảng khai báo AI** cụ thể: công cụ · việc · tệp/hàm · cách kiểm chứng (kể cả khi không dùng).
- [ ] Tag **v0.2 đã đẩy** và trang …/releases/tag/v0.2 mở được; tag trỏ vào commit mới nhất.
- [ ] Repo để **public** (giảng viên không nhận lời mời collaborator) — mở link ở cửa sổ ẩn danh để kiểm.
- [ ] Có docs/thay-doi-thiet-ke.md (hoặc một dòng “không có thay đổi so với BT1”) và docs/ai-disclosure.md.
- [ ] Không có .env thật và không có secret trong repo; có .env.example đủ biến.
- [ ] Đã nộp trên VLU E-learning và thấy trạng thái **đã nộp**.

Bốn mục tô đỏ là bốn lỗi hình thức hay gặp nhất mỗi học kỳ. Nộp sớm bản đủ điều kiện, rồi nộp lại bản cải thiện nếu còn thời gian — hệ thống lấy bản cuối.

KẾT THÚC BUỔI HỌC

Kết thúc Giai đoạn 2 — chuẩn bị Giai đoạn 3

Tự học, tự nghiên cứu

- Chưa nộp kịp tại lớp: nộp bù tối đa 3 ngày, trừ 10% điểm mỗi ngày trễ; quá 3 ngày tính 0 điểm và không đủ điều kiện dự thi.
- Hoàn thiện docs/bug-log.md: mọi lỗi đã gỡ trong Giai đoạn 2, mỗi lỗi đủ 7 dòng — BT3 yêu cầu ≥ 3 bug kèm link commit.
- Đọc tài liệu tự học buổi 13 theo track: công cụ kiểm thử của chuyên ngành.
- Đọc Sommerville (2015), Chapter 8 – Software Testing.
- Cài công cụ kiểm thử của track nếu chưa có; kiểm tra chạy được.

Buổi 13 (lý thuyết) — Bắt đầu Giai đoạn 3: Kiểm thử & Đảm bảo chất lượng

- Kim tự tháp kiểm thử: unit → integration → system → acceptance.
- Cách viết test case và Test Plan; tiêu chí thành công đo được.
- Phân loại bug Critical / Major / Minor.
- GitHub Actions: cấu hình CI chạy test tự động khi push.
- Phát ĐỀ BÀI BÀI TẬP 3 — hạn nộp cuối buổi 16.